

Практическая работа №4.

Построение графиков функций одной переменной на языке Python

Цель работы. Целью работы является изучение средств построения графиков функций одной переменной на Python.

Краткие теоретические сведения

Построение графиков является одним из основных инструментов наглядного представления данных в любых практических приложениях. Распространенной типовой задачей является построение графика функции одной переменной на заданном отрезке ее изменения. Функция при этом задается формулой, например, такой

$$y(x)=4e^{-x}\cdot\sin(18x). \quad (4.1)$$

В языке Python для визуализации графиков чаще всего используется библиотека matplotlib, содержащая огромное разнообразие методов для визуализации данных. В качестве иллюстрации типовых приемов ее использования можно порекомендовать [1].

В качестве примера рассмотрим задачу построения графика функции, заданной формулой (4.1), на отрезке изменения аргумента x от 0 до 6. Текст программы приведен в листинге 4.1.

Листинг 4.1. Программа, осуществляющая построение графика функции, заданной формулой (4.1)

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
x_min = 0
x_max = 6
x = np.linspace(x_min, x_max, 1000)
y = 4 * np.exp(-x) * np.sin(18 * x)
plt.figure()
matplotlib.rc('font', size = 11) # размер шрифта надписей
plt.plot(x, y) # собственно получение графика по данным
plt.xlabel("x")
plt.ylabel("y(x)")
plt.title("График функции  $y(x) = 4 * \exp(-x) * \sin(18x)$  на  
отрезке [0, 6]", \
        fontsize = 10) # размер шрифта названия графика
plt.xlim(x_min, x_max)
plt.grid() # накладывает сетку на график
plt.show() # отображение графика
```

Для генерации последовательности значений x в данном случае использована команда `linspace(...)`, при вызове которой третьим аргументом указывается количество точек, по которым будет построен график. Непосредственно визуализация графика происходит с помощью команды `plot` из импортированной в программе библиотеки `matplotlib.pyplot`. Для добавления обозначений для осей графика и добавления надписей используются `xlabel` и `ylabel`, для добавления названия диаграммы используется метод `title`. Для изменения размера шрифта меток на осях используется метод `rc` из модуля `matplotlib`. Этот метод позволяет управлять глобальными параметрами `matplotlib`, включая размер шрифта меток на осях.

Выбор количества точек является очень ответственным этапом: если взять малое количество точек, то график непрерывной функции окажется “угловатым”, фактически не отражая самой существенной особенности анализируемой функции. Если же выбрать слишком много точек, то построение графика может занять значительное время, особенно для функций двух аргументов; при этом после некоторого достаточно большого числа точек дальнейшее увеличение их количества не приведет к улучшению качества графика. В случаях, когда предварительный анализ свойств функции не позволяет заранее надежно определить, какое минимально-достаточное количество точек следует использовать в программе, можно действовать итерационно: удваивать количество используемых для построения графика точек до тех пор, пока изменения графика по сравнению с предыдущим построенным вариантом окажутся практически незаметными.

В результате запуска программы, текст которой приведен в листинге 4.1, будет выведено окно, в котором отображен график затухающей по экспоненте синусоиды со всеми запланированными программистом атрибутами (см. рис.4.1).

В листинге 4.2 приведен более сложный пример программы построения графиков функций [1], в котором осуществляется построение нескольких графиков функций на общей декартовой плоскости с использованием легенды, меток и дополнительной сеткой. Результатом работы данной программы будет график, приведенный на рис. 4.2. Комментировать программу и особенности полученного графика в данном случае, пожалуй, нет необходимости.

Еще один пример программы построения графиков функций приведен в листинге 4.3. Здесь, во-первых, можно получить наглядную иллюстрацию того, что точек для корректного представления графика бывает нужно очень много, во-вторых, показано, как задавать размер выводимого графика, в-третьих, проиллюстрирован принцип задания логарифмического масштаба (в данном случае – по оси абсцисс), который во многих случаях обеспечивает более информативное представление информации. Кроме того, в данном случае была определена пользовательская функция, которая осуществляет подготовку построения графиков с определенными программистом

атрибутами, а некоторые из параметров передаются в нее уже при вызове из основной программы.

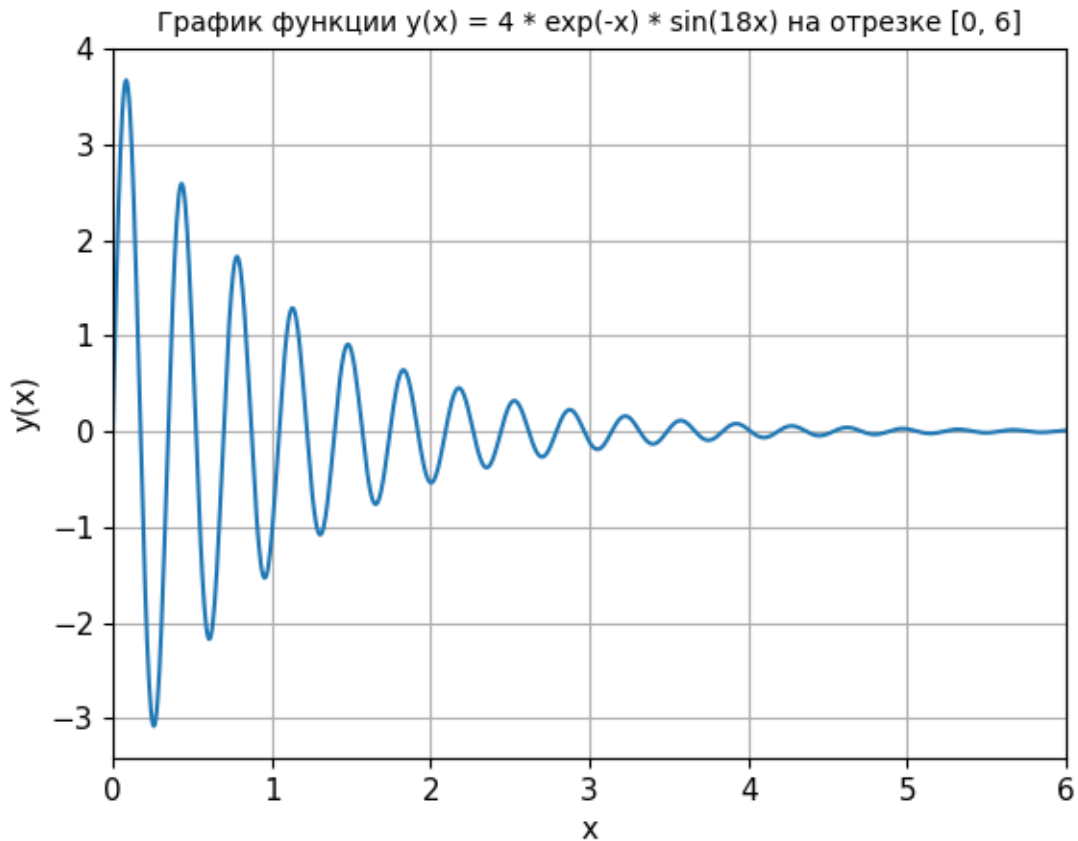


Рис. 4.1. График функции, построенный программой из листинга 4.1

Листинг 4.2. Программа, осуществляющая построение нескольких графиков функций на общей плоскости с необходимыми атрибутами

```
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

matplotlib.rcParams.update({'font.size': 12})

x = np.linspace(-1, 1, 1000)
plt.plot(x, x, label=r'$x$')
plt.plot(x, x**2, label=r'$x^2$')
plt.plot(x, x**3, label=r'$x^3$')
plt.plot(x, np.cbrt(x), label=r'$x^{1/3}$')
plt.legend(fontsize=16)
# включаем дополнительные отметки на осях
plt.minorticks_on()
plt.xlabel(r'$x$', fontsize=16)
plt.xlim([-1., 1.])
plt.ylim([-1., 1.])
# включаем основную сетку
plt.grid(which='major')
```

```
# включаем дополнительную сетку
plt.grid(which='minor', linestyle=':')
plt.tight_layout()
plt.show()
```

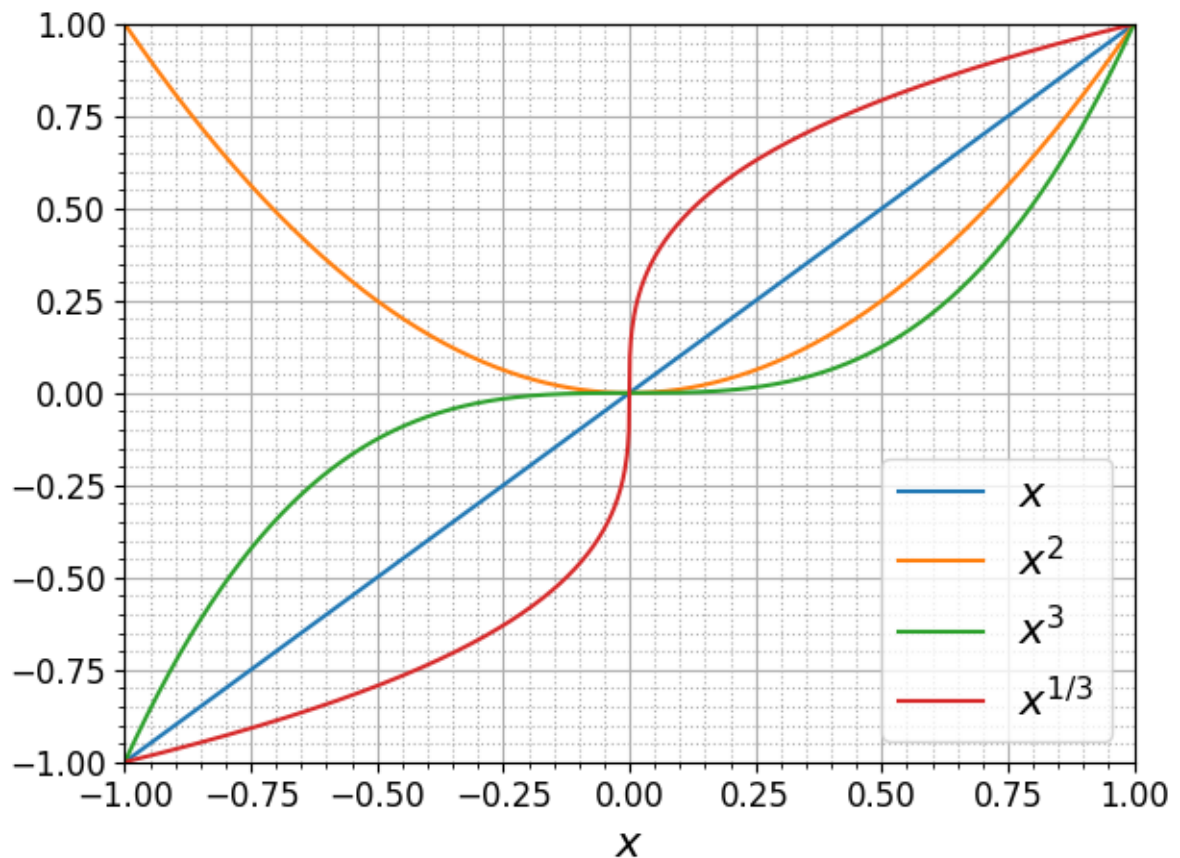


Рис. 4.2. График функции, построенный программой из листинга 4.2

Листинг 4.3. Построение графиков сложных функций в различных режимах

```
import numpy as np
import matplotlib.pyplot as plt
def graphs(points, x_scale):
    x = np.linspace(0.01, 5, points)
    y1 = np.sin(10 / np.abs(x))
    y2 = np.sin(4*x**3)
    plt.figure(figsize=(10, 2)) # размер окна 10 на 2 дюйм
    plt.plot(x, y1)
    plt.plot(x, y1, label="y1(x)")
    plt.plot(x, y2, label="y2(x)")
    plt.xlabel("x")
    plt.ylabel("y1, y2")
    if x_scale == 'log':
        x_scale = "лог."
        plt.xscale('log')
    else:
        x_scale = "лин."
    graph_title = "Графики функций y1(x) = sin(10/|x|) и y2(x) = sin(4*x^3), "+\

```

```

        str(points)+" точек, "+ x_scale + "масштаб"
plt.title(graph_title)
plt.legend(loc='best')
graphs(600,'lin') # точек мало
graphs(20000,'lin') # точек достаточно
graphs(20000,'log') # точек мало
graphs(500000,'log') # точек достаточно, ждать долго
plt.show() # ВНЕ ф-ции graphs, чтобы все три окна были сразу

```

Результатом работы данной программы будут 4 отдельных окна с графиками, которые приведены на рис. 4.3.

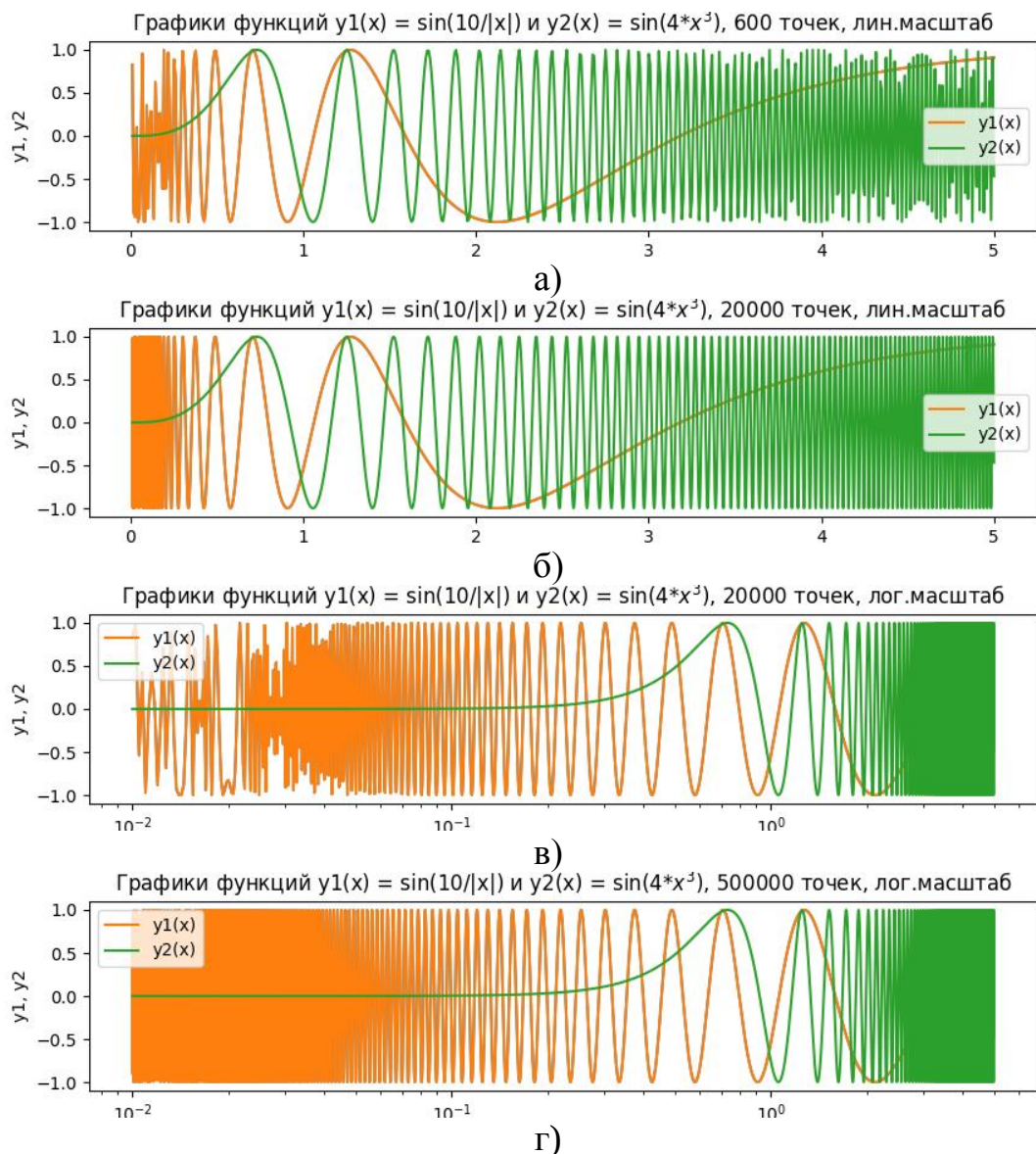


Рис. 4.3. Графики функций, построенных посредством программы из листинга 4.3: а) в линейном масштабе, 600 точек, б) в линейном масштабе, 20000 точек, в) в полулогарифмическом масштабе, 20000 точек, г) в полулогарифмическом масштабе, 500 тысяч точек.

Графики на рис.4.2, “а” и 4.3, “в” демонстрируют, что при недостаточном количестве точек в графике тригонометрических функций со сложным аргументом появляются особенности, которые не соответствуют ожидаемому поведению графика: он становится “рваным”, поскольку шаг по аргументу x в некоторых диапазонах становится слишком большим, из-за чего некоторые периоды колебаний функции просто пропускаются, в результате график на тех участках образуется по случайному набору данных.

Альтернативами программному способу вывода нескольких графиков, показанному в листинге 4.3, является использование либо метода `subplot` из библиотеки `matplotlib`, либо функции `subplot()` из библиотеки `pylab` (которую нужно вначале импортировать в программу).

Нужно иметь в виду, что основное назначение графика – это обеспечение возможности быстрого считывания значений функции для предусмотренных графиком значений аргумента. Если функциональная зависимость, график которой нужно построить, отличается большой вариабельностью значений в пределах заданного интервала аргумента то считывание данных из графика, построенного в линейном масштабе, становится затруднительным или даже невозможным. В этих случаях следует всегда прибегать к использованию полулогарифмического (когда логарифмический масштаб используется только по одной из осей графика) или логарифмического масштаба. Опробовав эти настройки, следует из множества полученных графиков выбрать тот результат, который окажется более наглядным с точки зрения представления особенностей функциональной зависимости.

Характерными примерами функциональных зависимостей, когда без использования полулогарифмического или логарифмического масштаба наглядность графиков теряется, являются:

- 1) гиперболические зависимости вида $f(x)=k/x^n$, графики которых строятся в относительно широком диапазоне изменения x , особенно в окрестности $x \approx 0$;
- 2) экспоненциальные зависимости вида $f(x)=k\exp(-x^n)$ и $f(x)=k\exp(x^n)$.

Даже если график строится не по значениям функции, которая задана формулой, а по некоторым данным, которые требуется визуализировать, нужно помнить о возможностях, которые предоставляет логарифмический масштаб. Общая рекомендация здесь следующая: когда график зависимости “прижимается” к одной из осей графика, использование логарифмического масштаба по другой оси часто позволяет улучшить наглядность графика.

Для иллюстрации вышеизложенного на рис.4.4 приведен пример построения графика функции $y(x)=1/x$ для x от 0.001 до 10 в линейном, полулогарифмическом и логарифмическом масштабе. Нетрудно видеть, что график на рис. 4.4, “а” совершенно неинформативен, - из него ни в одной точке нельзя визуально считать числовые значения с разумной точностью, а график на рис. 4.4”г” позволяет это делать с легкостью в любой точке, чему способствует наличие минорной сетки на графике (кстати, наличие сетки на

графике вообще является правилом хорошего тона, а при использовании логарифмического масштаба наличие сетки является практически обязательным, поскольку глазомер в этом случае работает плохо).

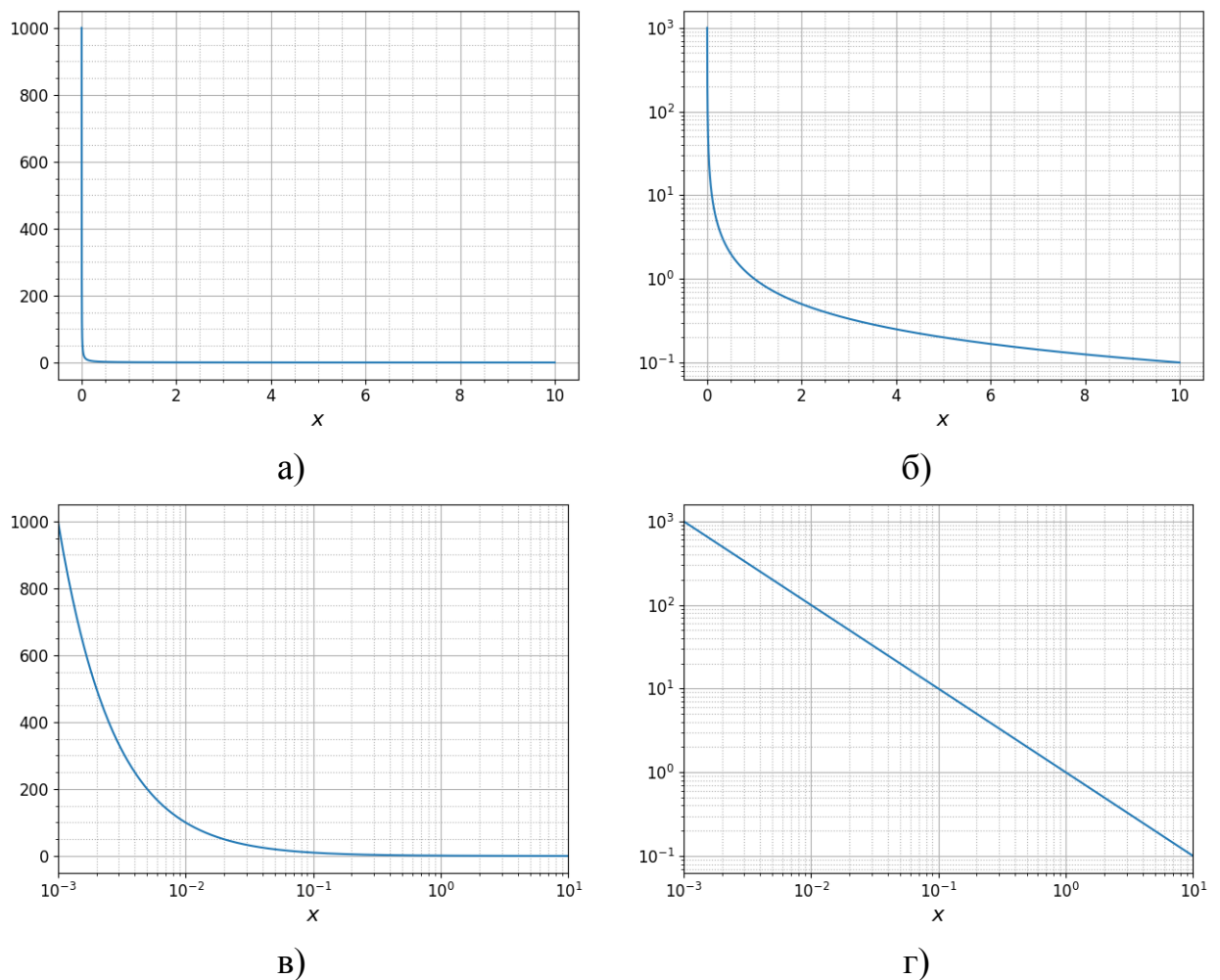


Рис. 4.4. Графики функции $y(x)=1/x$:

а) в линейно масштабе, б) в полулогарифмическом масштабе (по оси ординат), в) в полулогарифмическом масштабе (по оси абсцисс), г) в логарифмическом масштабе

Единственное ограничение, которое, к сожалению, не позволяет использовать логарифмический масштаб по одной или обеим осям графика – это наличие нулевых или отрицательных значений функции и/или ее аргумента. Хотя в Python это формальное ограничение можно обойти, используя масштабирование соответствующей оси с ключом ‘symlog’. На рис. 4.5 приведен пример построения графика функции $y(x)=\cos(10x)/(|x|+0.01)$ в диапазоне изменения x от минус 10 до 10 в линейном и логарифмическом масштабе с использованием ключа ‘symlog’ по обеим осям. Однако, нужно заметить, что данный подход, хотя и способствует лучшему визуальному восприятию деталей графика, в общем случае не обеспечивает возможности сколько-нибудь точного считывания

количественных данных из полученного графика в произвольно выбранной его точке, поскольку во-первых, в ближайшей к нулю окрестности используется линейный масштаб, который лишь при больших значениях переходит в логарифмический, а во-вторых, стандартные средства построения минорной сетки для логарифмического масштаба в этом режиме недоступны. В основном подход с масштабированием осей с ключом ‘symlog’ используется только для общей иллюстрации и качественной оценки поведения анализируемой функциональной зависимости. Вообще же, наиболее наглядное представление о поведении сложной функции часто удается получить при совместном рассмотрении графиков, построенных при различных способах масштабирования.

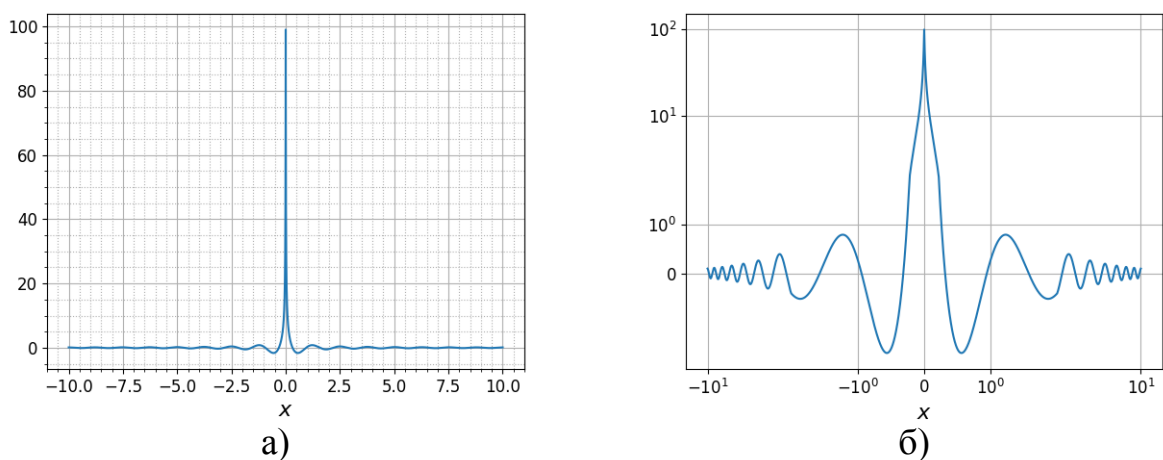


Рис. 4.5. Графики функции $y(x)=\cos(10x)/(|x|+0.01)$:

а) в линейном масштабе, б) в логарифмическом масштабе с ключом ‘symlog’

Более подробно с нюансами использования логарифмического масштаба можно ознакомиться на ресурсе [7].

В заключение можно отметить, что задачи наглядного графического представления данных весьма разнообразны и во многих случаях специфично связаны с конкретными практическими приложениями анализа данных, для понимания которых необходима соответствующая квалификация. Для предварительного ознакомления с некоторыми из таких средств можно порекомендовать ресурс <https://habr.com/ru/articles/468295/>. Дополнительно, для более глубокого погружения в теоретические вопросы, касающиеся библиотеки matplotlib можно ознакомиться с материалами ресурса <https://python-scripts.com/matplotlib?ysclid=m6582ggsra614956343>

Варианты заданий

Данная практическая работа предполагает выполнение трех заданий, для каждого из которых нужно написать программу на языке Python:

1) Задание №1: используя в качестве основы программу, написанную при выполнении практической работы №3, видоизменить ее таким образом, чтобы получить данные для построения графика зависимости абсолютной погрешности целевого вычисляемого значения от количества использованных членов ряда. Для вариантов, где нужно было вычислить значение не фундаментальной константы, а функции, для выполнения данного задания следует зафиксировать аргумент функции на каком-либо нетривиальном значении, например, рассчитывать значение $\sin(0.5)$ и считать это значение целевой константой. По полученным данным нужно построить соответствующий график. Финальное значение погрешности, которое должно быть отражено в графике – порядка 10^{-10} , если иное не будет обосновано объективно возникшими сложностями вследствие большого времени расчёта. В случае медленно сходящихся рядов при сохранении данных для построения графика рекомендуется использовать в программе прогрессивно увеличивающийся шаг по количеству членов ряда (т.е. вместо 1, 2, 3, 4, использовать, например, 1, 2, 4, 8, 16, и т.д.), поскольку в противном случае просматривается риск переполнения памяти.

2) Задание №2: построить графики функций $y_1(x)$ и $y_2(x)$ в диапазоне изменения аргумента x от ℓ до 10, где величина $\ell > 0$ должна быть рассчитана так, чтобы с *разумной* точностью выполнялось равенство $|y_2(\ell)| = \max(|y_1(x)|) = M$, $0 \leq x \leq 10$. Обучающийся самостоятельно принимает решение, каким образом он будет осуществлять нахождение ℓ : либо посредством теоретических расчетов, либо посредством программирования численных методов, основанных на постепенном подборе значения ℓ .

В результате выполнения задачи №2 нужно будет построить графики функций $y_1(x)$ и $y_2(x)$ совместно, причем таким образом, чтобы пределы вывода по оси ординат были бы ограничены величинами от минус M до M . Дополнительно хочется отметить, что для обозначения ℓ нежелательно в программе использовать строчную букву латиницы “l” из-за ее внешнего сходства с единицей; лучше использовать “L” или “eL”.

3) Задание №3: построить графики функций $f_1(x) = \sin(1000 \cdot y_1(x)/M)$ и $f_2(x) = \sin(1000 \cdot y_2(x)/M)$ в диапазоне изменения x от 0.01 до 10 сначала в линейном, затем в логарифмическом масштабе. В обоих случаях нужно подобрать такое количество точек, при котором построенные графики будут выглядеть корректно, аналогично рис. 4.3, “б” и “г”, без нехарактерных для гармонических функций разрывов (продемонстрированных на рис. 4.3, “а” и “в” в качестве примеров некорректного решения задачи).

При выполнении заданий обучающийся должен самостоятельно принять решение, какие настройки при построении графиков будут обеспечивать лучшую информативность и наглядность считывания данных.

Данные вариантов заданий приведены в таблице 4.1.

Таблица 4.1. Данные вариантов заданий

Вар.	Функция $y_1(x)$	Функция $y_2(x)$
1	$-4.1x^6+9.2x^5-11x^4+x-19$	$\frac{2}{x}+100x$
2	$3.6x^5-4.1x^4+9.2x^3-11x^2+x-19$	$-\frac{1}{3x}+500x$
3	$3.6x^6-4.1x^5+9.2x^4-11x^3+x-19$	$\frac{1}{x}+100x$
4	$x^6-4.1x^5-9.2x^3-11x^2+x+19$	$\frac{1}{x}+100\sqrt{x}$
5	$-3x^6+3.2x^5-x^4+x-15$	$-\frac{10}{x}+500x$
6	$x^6+8.6x^5-2.9x^4-7.2x^2+2x-14$	$100x-\frac{1}{\sqrt{x}}$
7	$1.2x^6-14x^5+5.2x^4-8.3x^3+x+2$	$\frac{1}{x^3}$
8	$-6.6x^6-3.4x^5-5.2x^3-2.7x^2+3x+19$	$100x-\frac{1}{10x}$
9	$-2.2x^6+6.2x^5-12x^4+8x^2$	$1000x-\frac{1}{2x}$
10	$5.6x^5-8.1x^4+7.2x^3-4x^2+x-12$	$100x+\frac{1}{\sqrt{x}}$
11	$3x^6-13x^5+8.2x^4-5x^3+20x$	$-\frac{1}{x^2}$
12	$-1.6x^6-5.5x^5-9.2x^3-6x^2+x+5$	$-\frac{100x-2}{x^2}$
13	$-2.5x^6+10x^5-8x^4+17x$	$\frac{100}{\sqrt{x}}-1000x$
14	$x^6+12x^5-4.9x^4-11x^2+9.2x-13$	$x-\frac{1}{x}$
15	$x^6-1.8x^5+1.2x^4-9x^3+5x-23$	$-\frac{1}{x}+100x$
16	$1.2x^6-2.4x^4-12x^3-10x^2+4$	$\frac{10}{x}+400x$
17	$-2x^6+5.2x^5-6.8x^4+x-19$	$\frac{1}{x}-10x$
18	$x^6+8.8x^5-5.3x^4+1.9x^3-7.4x^2+x+10$	$-\frac{10}{x}+100x$

Продолжение таблицы 4.1

19	$4.2x^6 - 3.3x^5 + 2.9x^4 - 2x^3 + x + 9$	$\frac{1}{10x} - 100x$
20	$-3.7x^4 - 20x^3 - 5x^2 + x + 21$	$\frac{1}{x^2} - x^2$
21	$-2.8x^6 + 4.6x^3 - x^2 - 20x$	$600x - \frac{1}{x}$
22	$x^6 + 3.9x^5 - 3.6x^4 + 5.7x^3 - 6x^2 + 7x - 8$	$\frac{x^2 - 1}{x}$
23	$x^6 - 6.7x^5 + 6.9x^4 + 8x^3 + x - 7$	$50x + \frac{1}{x}$
24	$x^6 - 2.8x^5 + 6x^4 + 7.1x^3 - x + 7$	$-\frac{1}{x}$
25	$-1.6x^6 + 9.9x^5 + 9x^4 + 6x - 19$	$500x + \frac{1}{x}$
26	$x^6 + 6.6x^5 - 3.2x^4 + 7.5x^3 - 3x^2 + x - 9$	$100x - \frac{1}{5x}$
27	$x^6 - 6.1x^5 + 8.2x^4 + 12x^3 + 12x$	$\frac{100}{x^3}$
28	$-4.5x^5 - 7.5x^4 - 11x^3 - 20x^2 + 2x + 1$	$\frac{10}{x^2}$
29	$-1.9x^6 + 8x^5 - 4x^4 + 4x - 1$	$\frac{100}{x}$

Порядок выполнения работы

1. Получить вариант задания у преподавателя и убедиться в правильном понимании задания и критериев его оценивания.
2. Последовательно написать и отладить программы для решения трёх, запустить их на выполнение в IDE и сохранить полученные результаты.
3. Получить результаты работы программ и подвергнуть их критическому анализу.

Требования к содержанию отчета

1. Цель работы.
2. Формулировка каждой полученной задачи и ее последующий анализ. Отчет следует разбить на подразделы (4.1, 4.2, 4.3 – для каждого задания отводится свой подраздел). Описание идей, положенных в основу определения величины ℓ (теоретический расчет или подбор программным способом).

3. Программы с необходимыми комментариями, а также описание наиболее важных ее фрагментов в основном тексте отчета.
4. Полученные графики при решении поставленных задач (4 рисунка: один для задания №1, два для задания №2 и один для задания №3). При желании обучающийся может привести и большее количество графиков, чтобы дополнительно проиллюстрировать особенности использования разных масштабов и настроек.
5. Выводы по работе.
6. Список использованной литературы.

При оформлении отчета о практической работе следует придерживаться требований правил, размещенных на сайте ГУАП.

Рекомендуемая литература

1. Язык Python. Визуализация данных с matplotlib // URL: <https://cpp-python-nisu.inp.nsk.su/textbook/sec4/ch8>
2. Python для анализа данных. Библиотека matplotlib // URL: https://mipt-stats.gitlab.io/courses/python/06_matplotlib.html
3. Python. Сборник упражнений / Стивенсон Б., пер. с англ. А.Ю. Гинько. – М.: ДМК-Пресс, 2021. – 238 с.
4. «Укус Питона» – «A Byte of Python» по-русски / Swaroop С.Н., пер. с англ. В. Смоляр. ver.2.02, apr 2020. – Sphinx. // свободно распространяется по лицензии Creative Commons Attribution-Share Alike 3.0 Unported, 2020.
5. Решаем задачи на Python / Джеймс Дэвис – Изд-во ЛитРес, 2024. – 158 с.
6. Введение в язык программирования Python / Чернышев С.А. – СПб: Изд-во ГУАП, 2023. – 236 с.
7. Делаем логарифмический масштаб у координатных осей. // URL: <https://proproprogs.ru/modules/matplotlib-delaem-logarifmicheskiy-masshtab-u-koordinatnyh-osey?ysclid=mc0djtil6900910909>